# AI-Powered Virtual Assistant for Real-Time Installation and Commissioning Support

A retrieval-based assistant that answers system integrators' questions from product documentation, with text and voice, on web and mobile, online or offline.

**Status** Draft for team review — no code written yet

**Scope of this document** System architecture, model selection, failure modes, safety guardrails, phased build plan

**Date** 9 October 2026

# 1. What we are building

An integrator is on site, halfway through installing a unit, and has a question. Today they stop work, open a 40-page PDF, and search it — or they call tech support and wait. We are building the thing they ask instead.

### The requirement, in one paragraph

A virtual assistant that answers installation and commissioning questions in real time from a knowledge base built out of product manuals, installation guides, wiring diagrams, FAQs and past issue resolutions. It takes text and voice. It runs on web and mobile. It works at sites with no internet connection. And when documentation is updated, integrators can upload the new version and the assistant stays current.

### Three words in the title that shape every decision

| Word | What it forces us to do |
|---|---|
| **Real-time** | Latency is a feature, not a nice-to-have. An integrator with a stalled job will not wait 20 seconds. Target: first words on screen in under 3 seconds. This is why we cache, why we prefer a small fast model over a large slow one, and why voice is push-to-talk rather than live streaming. |
| **Installation and commissioning** | A narrow, predictable domain. Not "ask anything about the product." Queries will be about wiring, fault codes, step sequences, parameter settings and part compatibility. We can legitimately refuse sales, warranty and general troubleshooting questions — and we can build a test set that covers the real domain. |
| **Support** | Not a replacement for tech support. Handing a hard question to a human is the system working correctly, not failing. This single idea drives the whole guardrail design in section 7. |

### What success looks like

Not "it answers everything." The target shape is:

- Answers roughly **85%** of in-scope questions
- Is roughly **99% accurate** on the ones it answers
- Says **"I don't know, here are the relevant pages"** on the rest

An assistant that answers everything at 90% accuracy sounds better and is worse. In this domain a single confidently wrong wiring answer ends the project. Abstention is a product feature and we should defend it as one.

> **The core design principle**
>
> The assistant never answers from the model's own knowledge. It answers only from text it has just retrieved from the documentation, and it shows the source for every claim. If it cannot find the source, it does not answer.

## 2. How it works, in plain language

The technique is called RAG — retrieval-augmented generation. The idea is simple once you see it: don't ask the AI what it knows, give it the right page and ask it to read.

### The wrong way, and why

You could train a model on the manuals. This fails three ways: it cannot cite sources, so nobody can verify an answer; it goes stale the moment a manual is revised; and models recalling facts from training produce plausible-sounding wrong ones — exactly the failure we cannot afford.

### The right way

Keep the documentation in a searchable database. When a question arrives, search for the few paragraphs most likely to contain the answer, paste those paragraphs into the model's prompt, and ask it to answer using only those paragraphs and to cite them. The model is doing reading comprehension, not recall.

**WORKED EXAMPLE — THE WHOLE IDEA IN SIX LINES**

**Integrator asks:** "What's the maximum screw head height when wall-mounting the terminal?"

1. **Search** the documentation for passages about wall mounting and screws.
2. **Find** three candidate passages; the best one is section 2.2 "Installation on a wall", page 8.
3. **Paste** that passage into the model's prompt along with the question.
4. **Model reads** the passage and writes: "Maximum 3 mm. Exceeding it risks damaging the electronics."
5. **Check** that "3 mm" actually appears in the passage. It does.
6. **Show** the answer with the citation: *Installation Instruction, §2.2, p.8.*

Everything else in this document is detail on how to do those six steps reliably, quickly, and safely when there are thousands of pages, ten people uploading documents at once, and a hundred integrators asking at the same time.

### Why this is mostly a retrieval problem, not an AI problem

Notice that steps 1 and 2 decide whether the answer is right. If the search returns the wrong passage, no model — however good — can produce a correct answer. If the search returns the right passage, even a small model can read it correctly.

So: **most of our engineering effort goes into retrieval quality, not the language model.** Teams who get this backwards spend weeks comparing models and ship a system that cannot find the right page.

> **What this means for the plan**
>
> We build and measure retrieval first, with no chat interface at all (phase 1). Only once it reliably finds the right passages do we add answer generation on top.

## 3. System overview

Seven layers. The important thing is not what is in each box, but that the boundaries between them are real — each boundary is a place where we change how a model is served without rewriting anything above or below it.

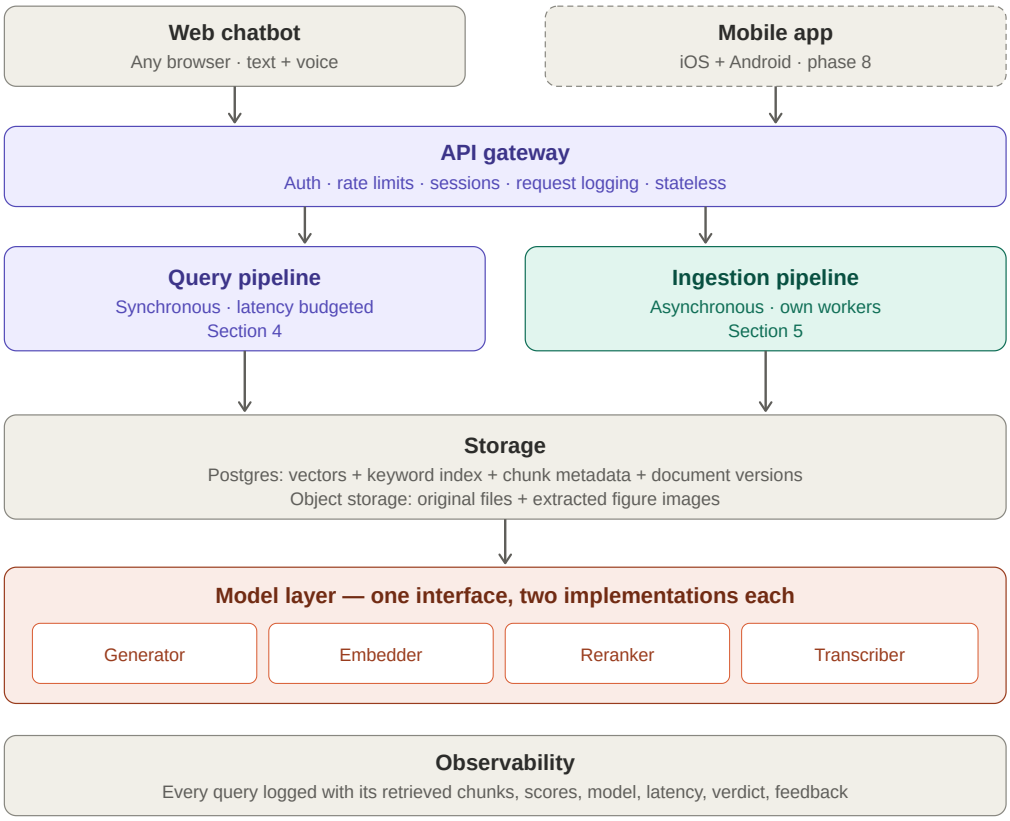



Figure 1 — Seven layers. The query and ingestion pipelines run on separate infrastructure and never compete for the same resources.

### Why the query and ingestion pipelines are separated

This is the single most important scalability decision in the design, and it is worth being explicit about why.

Ingestion is slow and heavy: parsing PDFs, running OCR on scanned pages, generating embeddings for hundreds of chunks. A query is fast and light. If they share a process or a machine, then the moment someone uploads a batch of documents, every integrator on site sees their answers slow to a crawl — precisely when the product is supposed to be real-time.

So: uploads go onto a queue and return immediately. A separate pool of workers drains that queue. The two pools scale independently. An upload spike can never degrade a live query.

## 4. The query path

What happens between an integrator pressing enter and an answer appearing. Ten stages, with a latency budget attached to each, because the total has to stay under about three seconds.

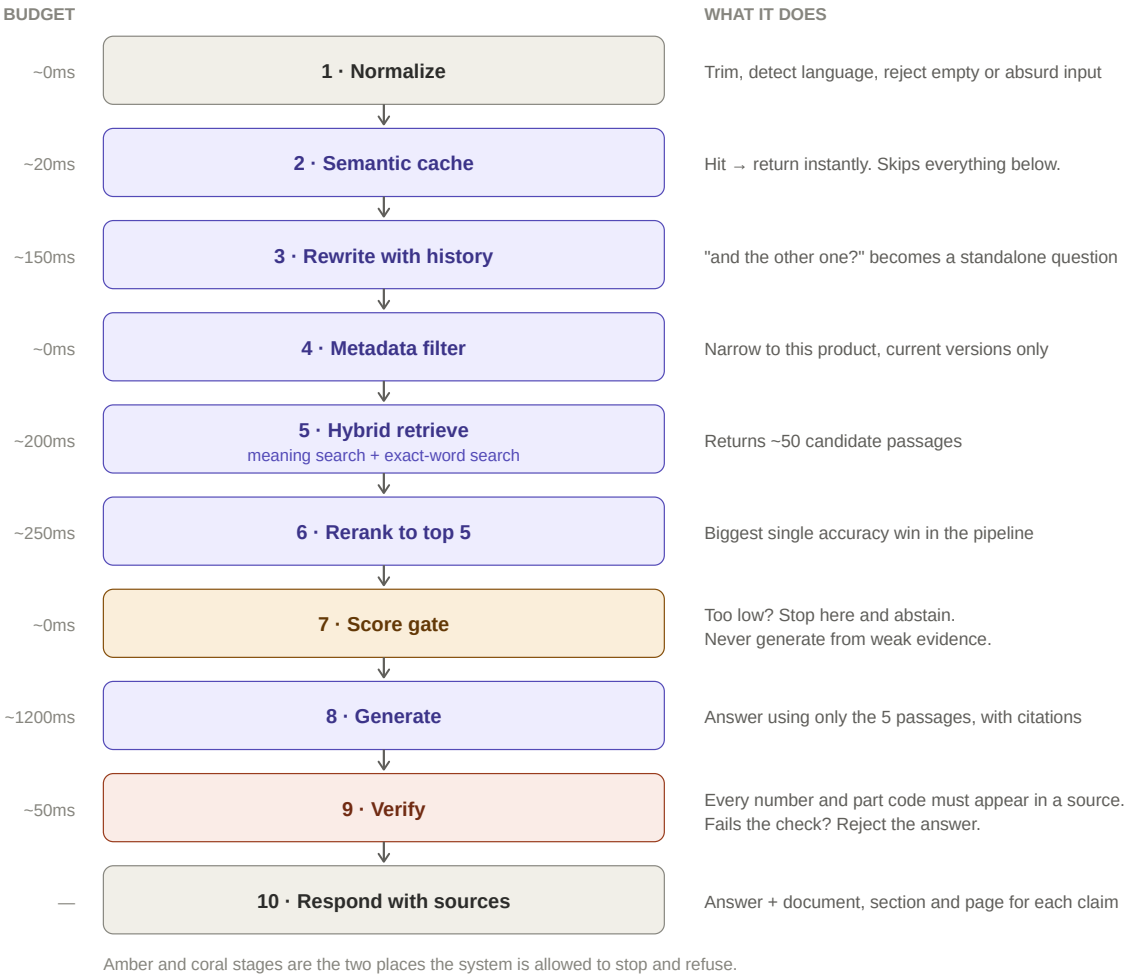



Figure 2 — The query path, with an indicative latency budget. Generation dominates, which is why a cache hit at stage 2 is worth so much.

### The three stages the first draft of this architecture was missing

| Stage | Why it has to be there |
|---|---|
| **Hybrid retrieve** (stage 5) | Technical documentation is full of exact strings: part numbers, fault codes, board revisions, terminal labels. Meaning-based search is bad at these — it understands "the mounting screw" but treats a part code as noise. So we run two searches and merge: one on meaning, one on exact words. |
| **Rerank** (stage 6) | The first search is fast and approximate; it casts a wide net of ~50 passages. A reranker is a slower, more accurate model that reads the question against each candidate and re-sorts them. Feeding the model the top 5 after reranking is dramatically better than the top 5 before it. Cheapest large accuracy win available. |
| **Verify** (stage 9) | Deterministic checks, not AI. Did the answer cite a document that was not in the retrieved set? Does every number in the answer appear in a source passage? These are cheap string checks and they catch the most dangerous failure — a confident invented value. |

### WORKED EXAMPLE — A QUESTION THAT GETS ANSWERED

**Query:** "What's the max screw head height for wall mounting?"

**Stage 2 — cache:** miss. First time anyone asked this today.

**Stage 4 — filter:** session says the integrator is working on this product line. Narrow to its documents, current revision only.

**Stage 5 — hybrid:** meaning-search finds passages about mounting and fixing; word-search finds passages containing "screw head". Merged: 50 candidates.

**Stage 6 — rerank:** the installation-on-a-wall section rises to rank 1 with a high score.

**Stage 7 — gate:** score is well above the floor. Proceed.

**Stage 8 — generate:** "Maximum 3 mm. A higher screw head can damage the electronics."

**Stage 9 — verify:** "3 mm" appears verbatim in passage 1 ✓. Cited document is in the retrieved set ✓. The passage's warning is attached to the answer ✓.

**Stage 10:** answer shown with source and page number.

### WORKED EXAMPLE — A QUESTION THAT GETS REFUSED

**Query:** "Can I use a third-party sensor from brand Y with this panel?"

Stages 1–6 run normally, but at **stage 7** the best reranked score is below the floor: nothing in the corpus discusses brand Y. The system stops before generation and replies:

> "I don't have documentation covering third-party sensors from that brand. The closest related material is the approved-accessories section — here is the link. For compatibility confirmation, contact technical support."

Without stage 7, a model handed five weakly-related passages will cheerfully invent an answer. This is the most common way RAG systems fail in production, and the mitigation is a threshold, not a better model.

## 5. The ingestion path

How documents get into the system — and how the system survives ten people uploading three files each at the same time.

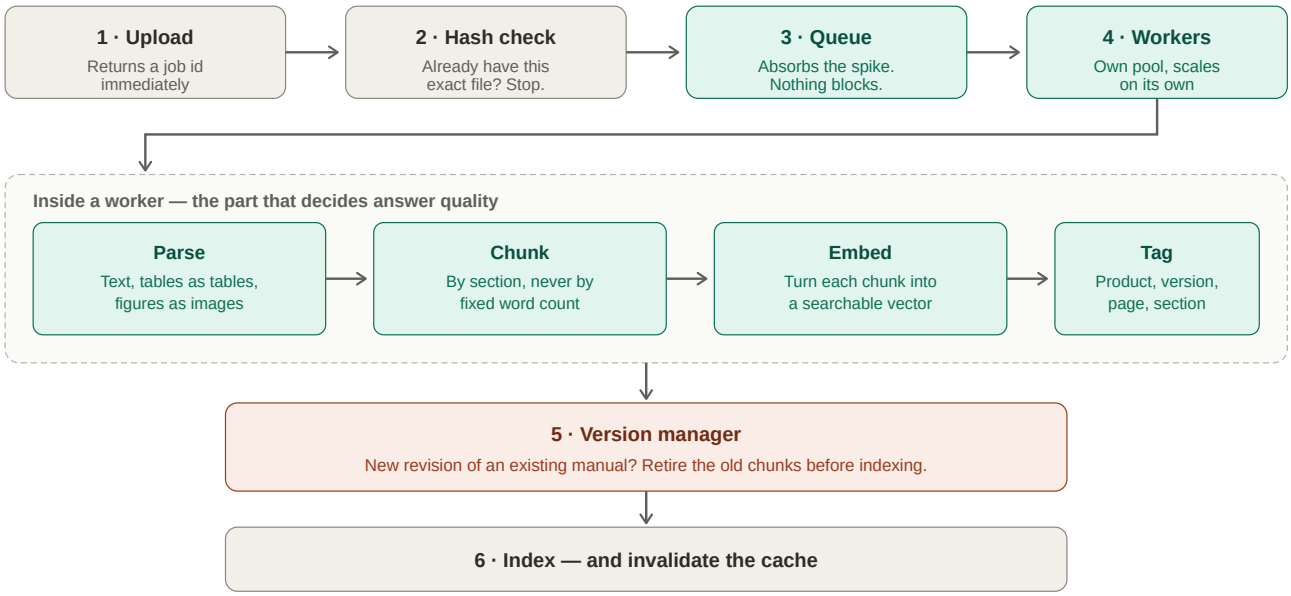



Figure 3 — Ingestion. Steps 3 and 4 are what make concurrent uploads safe; step 5 is what stops the assistant quoting a superseded manual.

### Why chunking is the part that decides answer quality

A "chunk" is one searchable unit of text. Chunking badly is the most common way to build a RAG system that retrieves correctly and still gives dangerous answers.

**✗ Naive: split every 500 words**

```
── chunk 41 ──────────────
...Step 3: Align the panel.
Step 4: Tighten the four
mounting screws and fit the
terminal into the housing.

── chunk 42 ──────────────
WARNING: Maximum screw head
height is 3 mm, otherwise the
electronics may be damaged.
Step 5: Connect the...
```

Someone asks "how do I mount it?" Chunk 41 is retrieved. It is the correct passage. The answer is complete, confident — and silently omits the warning that stops them destroying the unit.

**✓ Structure-aware: split by section**

```
── chunk: §2.2 Wall mounting ──
Steps 1–6 of the complete
procedure, with the 3 mm screw
head warning kept inline where
it appears.

metadata:
  section: 2.2
  page: 8
  type: procedure
  has_warning: true
```

The whole procedure travels together. A retrieval hit returns the steps and their warnings as one inseparable unit. The metadata lets us filter and cite precisely.

> **Hard rule for the implementation**
>
> A safety warning is never separated from the procedure it governs. If a chunk contains a step, it contains that step's warnings. We enforce this in the chunker and we test for it.

### Tables and diagrams

Two content types need special handling, and standard text extraction destroys both.

- **Tables** often hold the most answerable content in a manual — which board supports which function, which terminal does what. Flattened into a line of text, the row-to-column relationship is lost and the answer becomes wrong. We extract them as structured objects and index both the table and a generated prose description of it.
- **Wiring diagrams** are spatial information. OCR turns them into meaningless token soup. We extract the image, store it, index its caption and the surrounding prose, and **return the image itself alongside the answer.** Showing the integrator the actual diagram is more useful and more honest than describing it in words.

### Upload edge cases we design for now

| Case | Handling |
|---|---|
| Same file uploaded twice | Content hash → skip, don't re-embed |
| New revision of an existing manual | Supersede: retire old chunks, index new |
| Scanned, image-only PDF | Detect missing text layer → OCR path, flag low confidence |
| Corrupt or password-protected file | Fail that job cleanly with a readable reason |
| Very large file | Size cap, rejected at upload — not after ten minutes of work |
| Worker crashes mid-job | Jobs are idempotent and retryable; no half-indexed document is ever visible |
| Two of three files fail | Status is per file, not per batch — the user sees exactly which |
| We change embedding model later | Raw text and chunk boundaries are stored separately, so re-embedding is a batch job rather than re-parsing every PDF |

## 6. Model selection

Four model roles, all small open-weight models. Three of them run on our own machines for free; only the generator is served through an API, because it is the one role where CPU speed genuinely hurts. Total MVP cost: zero, or about $10 for convenience.

### The principle: run it locally unless there is a reason not to

The embedder, reranker and transcriber are all small enough to run on an ordinary laptop CPU. Downloading them from Hugging Face is free, there is no API key, no rate limit and no daily cap. Paying an API to run a 0.6B model we could host ourselves would be spending money to add a dependency.

The generator is different. A 4B–8B model on CPU produces a few tokens per second, which breaks the three-second target. That one role goes to an API with a free tier until we have hardware.

Two useful consequences. Our documents never leave our machines during phases 0 to 4, so the question of approval to send product documentation to a third party does not arise yet. And because every model here is open-weight, the offline deployment is the same models served locally — a config change, not a rewrite.

### Role by role ALL SLM

| Role | Model | Where it runs | Size | Why this one |
|---|---|---|---|---|
| **Embedder** makes text searchable | **BGE-M3** | **Local, CPU** free | ~2 GB | Dense and sparse retrieval in one model — the hybrid search we need without running two. Multilingual. Also on OpenRouter at ~$0.01/M tokens if we ever want it hosted. |
| **Reranker** re-sorts candidates | **BGE reranker** (m3 family) | **Local, CPU** free | ~1–2 GB | Pairs with the embedder. Biggest single accuracy gain in the pipeline, on modest hardware. |
| **Transcriber** voice to text | **NVIDIA Parakeet TDT 0.6B v2** faster-whisper as fallback | **Local, CPU** free | ~0.7 GB | Open weights — no NVIDIA API account needed. Small, fast, commercially licensed, and does not hallucinate during silence. Comparison below. |
| **Generator** writes the answer | **Qwen 3** via OpenRouter `:free` self-hosted 4B/8B later | **API** free tier | 3–6 GB if self-hosted | The one role where CPU is too slow. Our task is narrow — read 5 passages, answer, cite or abstain — which rewards instruction-following over deep reasoning. |

### What this costs

| Item | Cost | Detail |
|---|---|---|
| Embedder, reranker, transcriber | **Free** | Downloaded from Hugging Face and run locally. No account tier required for model downloads. |
| Generator, OpenRouter free tier | **Free** | 20 requests/minute, 50 requests/day |

| | | |
|---|---|---|
| Optional: raise the daily cap | **~$10 once** | Purchasing ≥10 credits raises the free-model allowance from 50 to 1,000 requests/day. Recommended — 50/day is restrictive while developing. |
| Embedding via API, if ever needed | ~$0.0025 | BGE-M3 at $0.01/M tokens; a ~350-page corpus is roughly 250k tokens. Not a budget line. |

**Two things to watch on the free tier**

A negative OpenRouter balance returns 402 errors *even on free models*, so keep the balance at or above zero. And 20 requests/minute is comfortable for two developers but nowhere near a hundred-concurrent-user demo — before showing this to anyone at scale we need paid inference or our own serving. Hugging Face's own Inference API is not an alternative: free accounts get no monthly credits at all.

**What this means for hardware**

No GPU is required for phases 0 to 6. What matters is RAM: roughly **8 GB free** runs the embedder and reranker comfortably, **16 GB** is comfortable with voice as well. A GPU only becomes necessary at phase 7, when the generator moves in-house for offline deployment.

## Speech-to-text: the options compared

Worth going through in detail, because the obvious choice (Whisper) is probably not the right one when the transcriber has to share a laptop CPU with the embedder and reranker.

| Model | Size | English WER | Speed | Languages | Streaming | Licence |
|---|---|---|---|---|---|---|
| **Parakeet TDT 0.6B v2** | 600M | **~6.05%** | RTFx ~3,386 | English only | No | Open, commercial use |
| **Parakeet TDT 0.6B v3** | 600M · 680MB INT8 | 6.34% | ~3,332× real time | 25 — EU, EN, RU, auto-detect | No | CC-BY-4.0 |
| **Parakeet Unified EN** | 600M-class · 631MB | **5.91%** | not stated | English only | No | not stated |
| **Nemotron Speech Streaming EN** | 600M · ~640MB | 6.93% streaming | not stated | English only | **Yes** — 80ms–1.12s | OpenMDW |
| **Nemotron 3.5 ASR** | 600M · ~640MB | not stated | not stated | 40 locales, auto-detect | **Yes** | OpenMDW |
| **Canary 1B / 1B Flash** | 1B | — | — | multilingual | No | check per model |
| **Whisper large-v3** | 1.55B · 3GB | ~7.4% | well under 1/10th of Parakeet v3 | **99** | No — 30s windows | **MIT** |
| **Whisper large-v3-turbo** | 1.6GB | — | faster than large-v3 | 99 | No | MIT |

**On the Whisper variants:** `faster-whisper`, `WhisperX` and `distil-whisper` are the same model re-implemented, not different models. faster-whisper is substantially quicker at the same accuracy, WhisperX adds word alignment and speaker diarization, distil-whisper is smaller and faster with an English focus.

### What actually separates them for this project

- **Footprint.** Parakeet is roughly 6× smaller than Whisper large-v3 and far faster. On an ordinary CPU already running the embedder and reranker, that is the difference between transcription being effectively free and it blocking everything else.
- **Silence behaviour.** Parakeet's architecture avoids generating text when nothing is being said — a known Whisper failure mode, and relevant on a noisy site.
- **One WER point is not the real story.** Word error rate weights every word equally, but a wrong filler word costs nothing and a wrong part number costs an hour. Whichever model we pick, showing the transcript back for confirmation before answering does more for accuracy than the model choice does.
- **Whisper's advantage is breadth and maturity** — 99 languages against 25, an MIT licence, and years of deployment recipes. Parakeet v3 omits Chinese, Japanese and Korean entirely.
- **Energy.** A third-party benchmark cited in one comparison claims Parakeet uses roughly 8–10× less energy per hour of audio at comparable quality. Directional, not verified.

### Which to pick

| Scenario | Model |
|---|---|
| English-only sites, push-to-talk — **our MVP** | **Parakeet TDT 0.6B v2** — best English accuracy at the smallest footprint |
| European languages needed | **Parakeet TDT 0.6B v3** — 25 languages for 0.3 WER points |
| Asian languages needed | **Whisper large-v3** — Parakeet does not cover CJK |
| If we ever move from push-to-talk to live streaming voice | **Nemotron Speech Streaming** — the only streaming option here |

> **Two caveats before treating this as settled**
>
> These figures come from vendor blogs and comparison sites that have their own products to sell, and benchmark conditions differ between rows — the speed numbers especially are not strictly comparable. Separately, OpenMDW is a newer licence: if we go the Nemotron route it needs reading properly rather than assuming it is permissive.

> **Why Qwen 3 8B rather than a larger local model**
>
> A 14B-class model such as Phi-4 reasons better but doubles the VRAM floor to buy capability this task does not use. Qwen 3 also ships at 0.6B / 1.7B / 4B, so weaker on-site hardware runs a smaller sibling with the same prompts. Phi-4 is the upgrade path if we measure 8B as the bottleneck — not an assumption to start from.

> **The one decision that is expensive to reverse**
>
> Different embedding models produce incompatible vectors, so changing one means re-embedding the whole corpus. We store extracted text and chunk boundaries separately from the vectors from day one — then a model switch is a batch re-embed, not a re-parse of every document.

## Serving: how 100 concurrent users is actually handled

Concurrency is an infrastructure question, not a model question. Four things matter:

- **The free tier is not the answer.** 20 requests/minute is right for two developers and nowhere near a hundred concurrent users. Before any scale demo we move to paid inference or our own serving — a config change, not a rebuild.
- **Development can also use Ollama** — one command to run a model locally with an API to build against. It serves one request at a time: fine for building, wrong for production.
- **Production uses vLLM**, which batches concurrent requests on shared hardware. Again a config change, provided everything speaks an OpenAI-compatible interface — which we enforce.
- **The cache does the heavy lifting.** The same questions recur constantly across integrators on a job. A cache hit costs ~20ms instead of ~1.5s, which changes the concurrency maths more than any serving tweak — and it costs no API quota at all.

### Deployment

Everything except the generator runs on the same machine as the application, so the whole system is one deployable unit — the same stack on a developer laptop, a demo box, or a customer site. The only difference at a customer site is that the generator moves in-house too.

| Component | Runs on | Note |
|---|---|---|
| Web frontend | Same machine | No external hosting needed; works on an isolated network |
| API backend + workers | Same machine | Query pool and ingestion pool stay separate processes even on one box |
| Postgres + pgvector | Same machine | Vectors, keyword index, metadata and versions in one database |
| Embedder, reranker, transcriber | Same machine, CPU | No GPU, no API, no keys |
| Generator | OpenRouter API<br>→ local server at phase 7 | The only external dependency in the system |

## 7. Safety guardrails

The assistant gives guidance to someone installing equipment. A wrong answer has physical consequences. These are the layers that stop one.

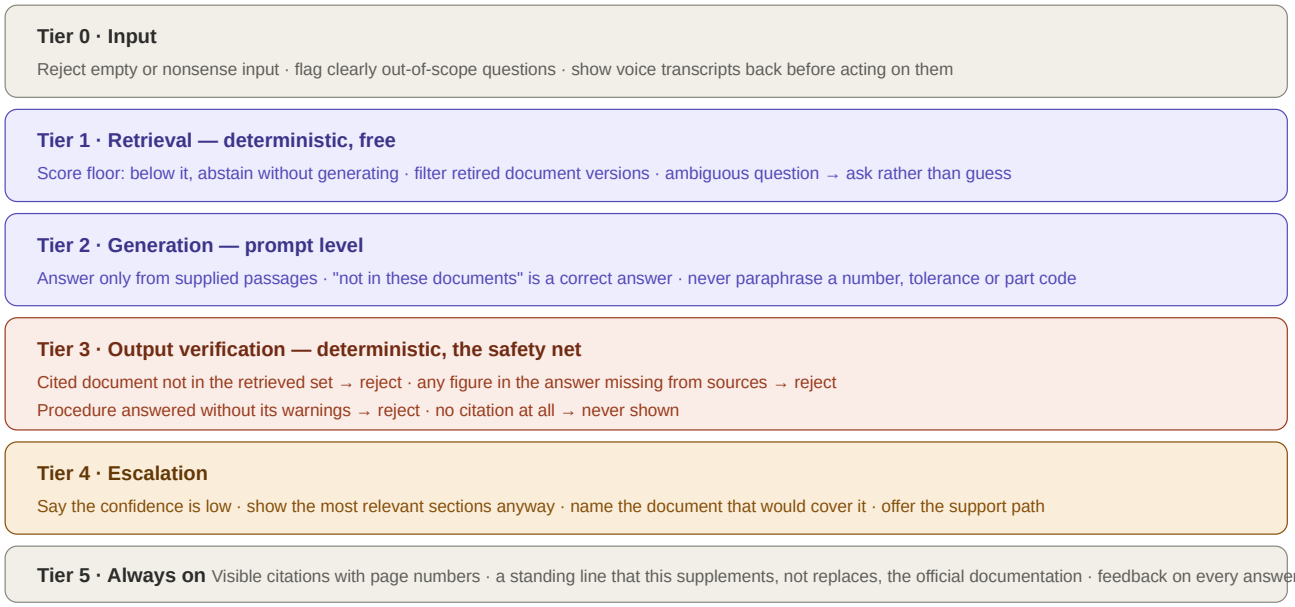



Figure 4 — Six tiers. Tiers 1 and 3 are plain code, not AI, and they catch the most dangerous failures.

Note how little of this is machine learning. The checks that matter most — does every number in the answer appear in a source, was the cited document actually retrieved, did a procedure lose its warnings — are string comparisons. They are cheap, they are testable, and they cannot themselves hallucinate.

### The failure modes we are designing against

| # | Failure | Mitigation |
|---|---|---|
| 1 | Retrieves a plausible but wrong passage | Reranker, then score floor, then abstain |
| 2 | Part number matches nothing by meaning | Hybrid search with exact-match boost |
| 3 | Answer cites a superseded manual | Version metadata; retired chunks filtered out by default |
| 4 | Correct answer depends on firmware version | Detect version-dependent content; ask, or caveat explicitly |
| 5 | Model invents a part number or a value | Tier 3: every identifier must appear verbatim in a source |
| 6 | Model cites a document that was never retrieved | Tier 3: reject the answer outright |
| 7 | Table flattened, numbers scrambled | Structured table extraction; tested against known values |
| 8 | Question needs a document we don't have | Detect cross-references; say what is missing by name |
| 9 | Follow-up question loses its referent | Session memory plus query rewriting (stage 3) |
| 10 | Thirty files uploaded at once | Queue plus separate worker pool; never inline |
| 11 | Worker dies mid-job | Idempotent, resumable jobs; no partial documents visible |

| | | |
|---|---|---|
| 12 | Voice garbles a part number | Show the transcript for confirmation before answering |
| 13 | Noisy site audio | Push-to-talk, not live streaming; confidence score on transcript |
| 14 | Generator API unavailable or rate-limited | Fall back to a self-hosted model if present, then to retrieval-only results |
| 15 | Cache serves a stale answer after re-indexing | Invalidate the cache on every index change |
| 16 | Latency spikes under load | Separate pools, backpressure, an explicit "busy" rather than a silent timeout |
| 17 | Embedding model swapped, index inconsistent | Version the index; block queries against a mixed-model index |
| 18 | Retrieval quality degrades silently over time | The evaluation set, re-run on every change |

## Degraded mode

If generation is unavailable — no connectivity, quota exhausted, no local generator — the system should still return the retrieved passages with their citations. An integrator who is shown the right section of the right manual has been helped, even with no generated answer. The system should never return nothing.

> **The fallback chain**
>
> Generator API → self-hosted model → retrieval-only passages. Three levels, each still useful. Note that the embedder and reranker are local, so retrieval keeps working with no connectivity at all — only the written answer is lost.

## 8. Build plan

Nine phases. Phases 0–4 are the MVP; everything after satisfies the remaining requirements in the scope document.

| # | Deliverable | What it proves |
|---|---|---|
| 0 | **Evaluation question set**<br>80–120 real questions with known answers | That we can measure anything at all. Every later change is judged against this. |
| 1 | **Ingestion + hybrid retrieval**<br>No interface, no AI answers | That we can find the right passage. The hardest part of the project. |
| 2 | **Add the reranker**<br>Measure the before/after delta | That reranking earns its latency cost. Measured, not assumed. |
| 3 | **Generation + citations + tiers 1–3** | That answers are trustworthy and refusals happen when they should. |
| 4 | **Web chat interface** | Demoable. This is the MVP. |
| 5 | **Async upload pipeline** | That concurrent uploads don't degrade live queries. |
| 6 | **Voice input**<br>Push-to-talk | Hands-free use on site. |
| 7 | **Self-host the generator**<br>The only role still on an API | The offline / on-premise requirement. Needs a GPU; everything else is already local. |
| 8 | **Mobile app** | Full scope satisfied. Mostly a UI over the same API. |

> **Do not skip phase 0**
> Every team building a system like this skips the evaluation set, then spends the following weeks arguing about whether a change helped. Writing 100 questions with known answers takes about a day and it is the highest-value day in the project. It is also what lets us make an accuracy claim to the customer rather than an assertion.

### What "production-ready MVP" means here

Not more features. These five properties:

1. **A measurable accuracy number** from the evaluation set, re-run on every change.
2. **Full query tracing** — for any answer, we can reconstruct exactly which passages produced it and why.
3. **Defined behaviour when a dependency fails** — every external call has a fallback.
4. **Idempotent ingestion** — re-running it is always safe.
5. **Config-driven model selection** — API to self-hosted is a config file, not a branch.

### A caution on scope

Everything here is the target architecture; we should not build all of it for the MVP. The boundaries are what is expensive to retrofit — the queue, the separate pools, the model interface, version metadata on chunks — so those go in early even in trivial form: one worker instead of a pool, tier 1 and 3 checks only. This kind of project usually fails by building the complete system and never shipping the demo. We will be judged on a working chat interface answering real questions correctly.

## 9. Open questions

Decisions this document cannot make. Each one changes something concrete, so they are worth closing before or during phase 1.

| # | Question | What the answer changes |
|---|---|---|
| 1 | **Does "100 users at once" mean 100 concurrent requests, or 100 registered users?** | Enormously different. The first needs real GPU serving and batching; the second runs comfortably on one modest machine. |
| 2 | **Is there approval to send retrieved passages to a third-party model API?** | Less urgent than it was: the embedder, reranker and transcriber are local, so whole documents never leave our machines. Only the handful of passages sent to the generator do. If even that is not permitted, we self-host the generator earlier and need a GPU sooner. |
| 3 | **What hardware will on-premise sites actually have?** | VRAM is the hard constraint and it picks the model size: 4B, 8B or 14B. |
| 4 | **How large is the full document corpus, and how often is it revised?** | Decides whether Postgres is sufficient or we need a dedicated vector database, and how much the version manager matters. |
| 5 | **How does an offline site receive updated documentation?** | Needs a defined mechanism even if the first answer is "an export bundle on a USB drive". Currently unspecified in every version of this architecture. |
| 6 | **Should all integrators see all documents?** | If not, we need access control as metadata filtering, which is far easier to design in now than to add later. |
| 7 | **Which model licences are acceptable for commercial use?** | "Open weights" is not the same as "free for commercial use". Worth confirming per model before we build on one. |

### Known weak points in this design

Two things in this document are genuinely unresolved rather than merely undecided, and should be flagged honestly rather than left to look solved:

**Wiring diagrams.** "Parse, OCR, chunk" makes image-based schematics sound handled. They are not. A wiring diagram is spatial information that OCR flattens into meaningless text. The realistic options are a vision model that describes each diagram at ingestion time, or returning the diagram image itself alongside the text answer. The plan assumes the second, because showing the integrator the actual diagram is both more useful and more honest than describing it.

**Offline synchronisation.** No version of this architecture yet shows how an air-gapped site receives revised documentation. It needs a defined mechanism. Leaving it blank is how it gets discovered during a customer deployment.

---

Prepared as a review draft. The purpose of this document is to be argued with — particularly sections 6 and 9, where the decisions are closest to reversible and the open questions are cheapest to close now.

Speech-model figures in section 6 are drawn from NVIDIA's own published benchmarks and from two third-party comparisons (openwhispr.com, bibigpt.co). Model availability, pricing and benchmark positions move monthly — re-check before committing to any of them.